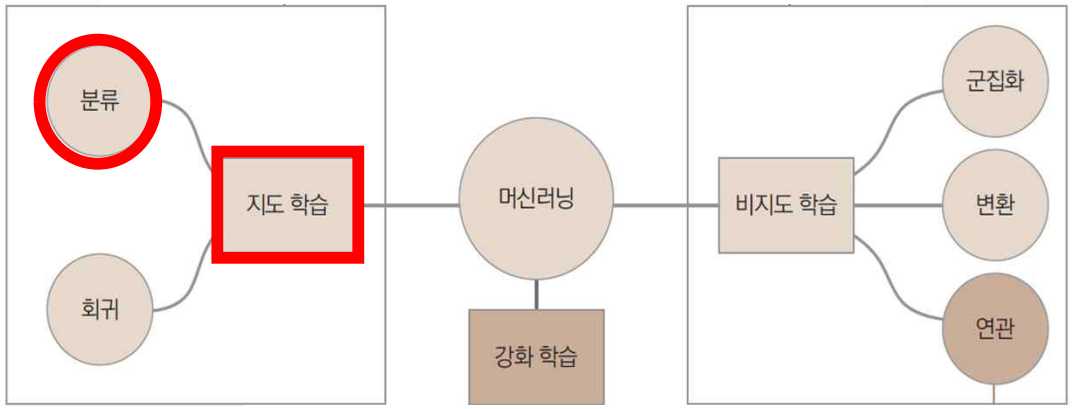
머신러닝 프로그래밍
Machine Learning Programming

Lecture 3 : 분류

충북대학교
CHUNGBUK NATIONAL UNIVERSITY
충북대학교
산업인공지능연구센터

머신러닝 기법 : **지도학습**, 비지도학습, 강화학습



- 회귀 : 연속된 값을 측정하는 문제
  (ex) 중고차 가격 예측
- **분류 : 범주형 값을 측정하는 문제**
  (ex) 마케팅 문자를 보냈을 때 구매할 것 같은 고객을 예측하는 문제는 Yes/No 두 가지 범주로 표현

- 군집화(clustering) : 비슷한 데이터끼리 묶어주는 방법
  (ex) 소비 패턴이 유사한 카드 고객을 묶음.

- 변환(transformation) : 목적에 따라 데이터를 다른 형태로 바꿈.
  (ex) 변수 100개의 특성을 최대한 살려 변수 2개로 압축

- 연관(association) : 일명 장바구니 분석이라고도 하며, 유사한 구매 목록을 가진 고객끼리 비교하여, 서로의 장바구니에 담기지 않은 새로운 아이템을 추천을 하는 목적 등으로 사용.

분류(Classification) vs 회귀(Regression)

구분	분류 (Classification)	회귀 (Regression)
목표	데이터가 어떤 카테고리(범주)에 속하는지 예측 Which one ?	데이터의 연속적인 수치(값)를 예측 How much ?
결과값	이산적	연속적
예시	스팸 메일 차단, MNIST(0~9 숫자판별)	주가예측, 기온예측
평가방법	정밀도, 재현율, ROC	MSE(평균제곱오차), R^2 Score
문제확장	다중분류(3개이상)	다중회귀 (입력 특성이 여러 개)
핵심 모델	로지스틱 회귀(3,4주차), 의사결정트리(9주차) K-최근접이웃(6주차), 서포트벡터벡신(7주차)	선형회귀(5주차), 릿지(Ridge) & 라쏘(Lasso) 엘라스틱 넷 (Elastic Net)

✓ 로지스틱 회귀 : 작동원리가 선형회귀와 거의 유사, but 시그모이드(sigmoid) 함수에 통과시켜 0과 1사이의 확률값으로 변환하하여 분류하기 때문에 분류로 봄

로지스틱 회귀

- 분류 문제 : 몇 가지 이산적 값 중 하나를 선택하는 모델. ‘분류 모델’이라고 부름

표1. GRE와 GPA 정보를 활용하여 합격 여부를 나타내는 데이터

Number	Admit	GRE	GPA	Number	Admit	GRE	GPA
1	0	380	3.61	16	0	660	3.34
2	1	660	3.67	17	1	740	4.00
3	1	800	4.00	18	0	560	3.19
4	1	640	3.19	19	0	380	2.94
5	0	520	2.93	20	0	400	3.65
6	1	760	3.00	21	0	600	2.82
7	1	560	2.98	22	1	620	3.18
8	0	400	3.08	23	0	560	3.32
9	1	540	3.39	24	0	640	3.67
10	0	700	3.92	25	1	680	3.85
11	0	800	4.00	26	0	580	4.00
12	0	440	3.22	27	0	600	3.59
13	1	760	4.00	28	0	740	3.62
14	0	700	3.08	29	0	620	3.30
15	1	700	4.00	30	0	580	3.69

로지스틱 회귀

- [표 1]은 GRE와 GPA 데이터를 통해 대학의 합격 여부 (Admit 열)를 나타냄
- GRE와 GPA 정보를 산점도로 표현
  - 합격자는 파란색, 불합격자는 빨간색으로 나타냄

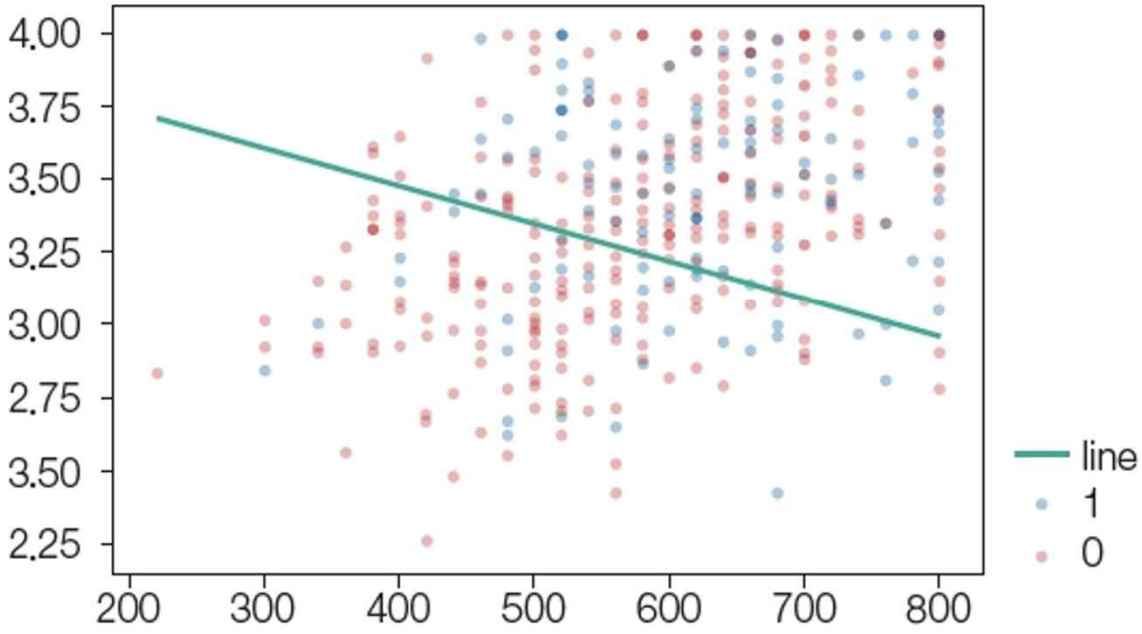


그림 1. 산점도 위에 선형회귀 모델 표현

로지스틱 회귀

- 초록색 선을 추가해 선 상단은 합격, 선 하단은 불합격
  - 아래 수식으로 기존 선형회귀 모델을 적용

$$f(x) = 4 - 0.0013 \times GRE - GPA$$

$$Admit = \begin{cases} 1 & \text{if } f(x) \geq 0 \\ 0 & otherwise \end{cases}$$

  - 문제점:

❶ $f(x)$ 의 값이 1 이상이나 0 이하로 나올 수 있음

❷ 각 피쳐들이 Y에 영향을 주는 것을 해석하는 문제

❸ 사건의 발생 여부는 이산적인데 실제 $f(x)$ 수식은 연속적

1. 로지스틱 회귀의 개념

- 이진 분류(binary classification) 문제를 확률로 표현
- 어떤 사건이 일어날 확률을 P(X)로 나타내고 일어나지 않을 확률을 1 - P(x)로 나타냄 (0 ≤ P(X) ≤ 1)
- 오즈비(odds ratio) : 어떤 사건이 일어날 확률과 일어나지 않을 확률의 비율

$$\frac{P(X)}{1-P(X)}$$

로지스틱 회귀

- 확률이 올라갈수록 오즈비도 급속히 상승

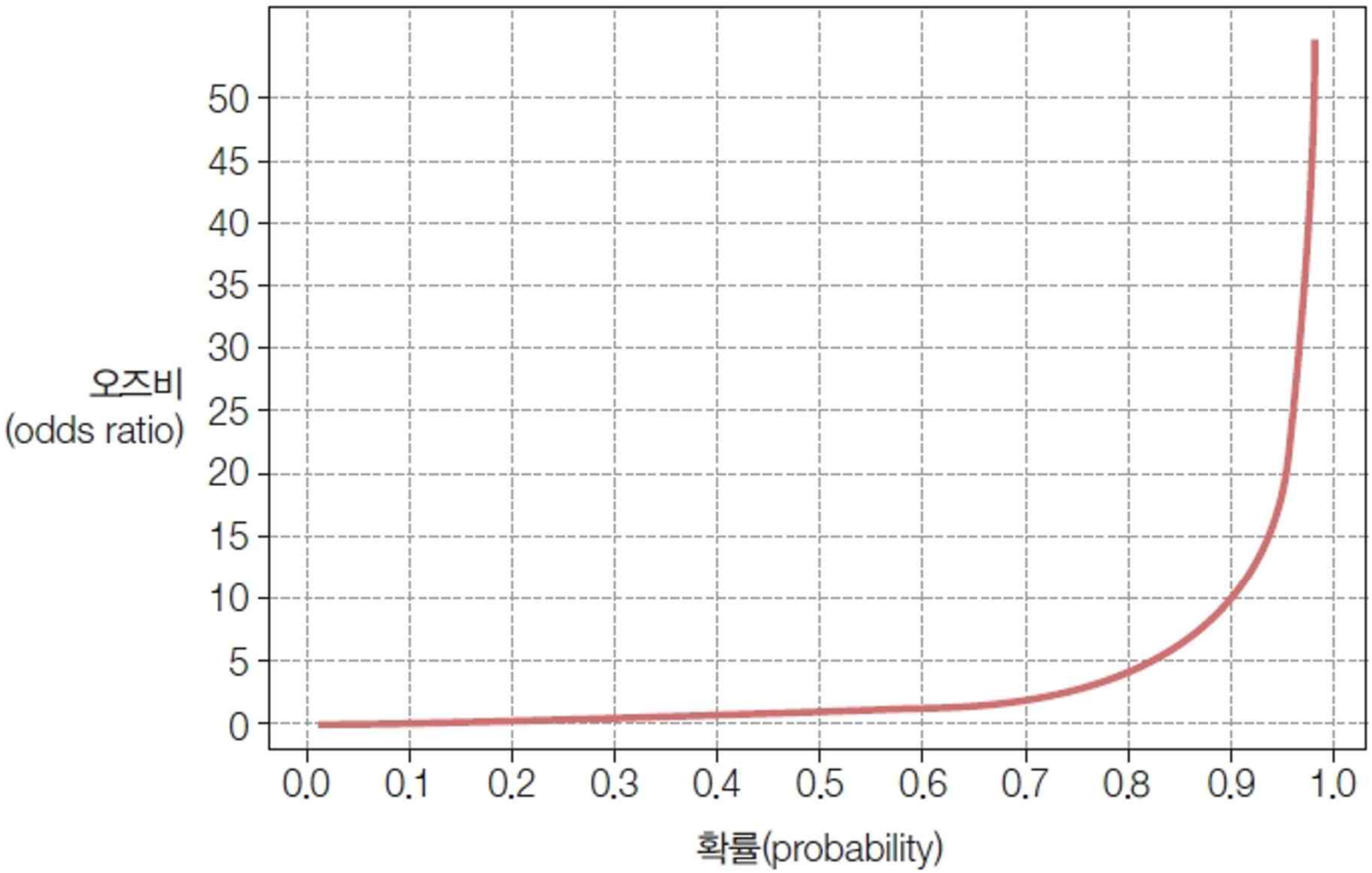


그림2. 확률이 올라가면서 오즈비가 상승하는 그래프

로지스틱 회귀

- 로짓(logit) 함수 : 오즈비에 상용로그를 붙인 수식

$$
\begin{aligned}
logit(p(y=1|x)) &= \log_e\left(\frac{p}{1-p}\right) \\
&= \log_e(p) - \log_e(1-p) \\
&= -\log_e\left(\frac{1}{p}-1\right)
\end{aligned}
$$

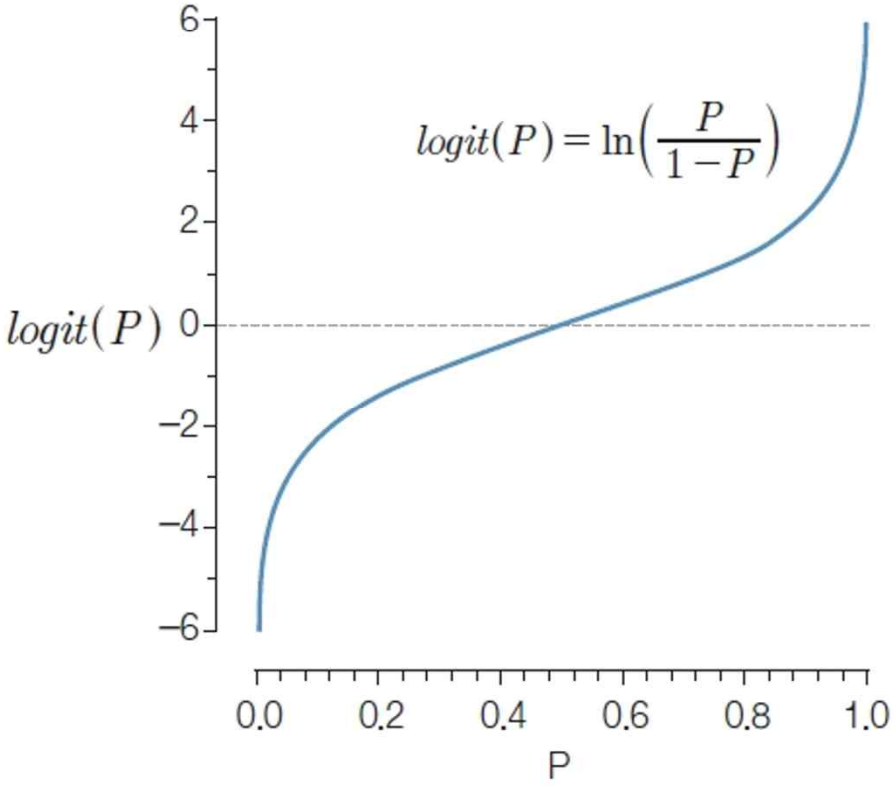


그림3. 로짓 함수 그래프

로지스틱 회귀

로짓(Logit)에서 시그모이드(Sigmoid) 함수 도출하기

1. 로짓 함수 정의 (오즈에 자연로그 취하기) 우리가 예측하고자 하는 선형 방정식($ax + b$)을 로그 오즈와 같다고 둡니다.

$$\ln\left(\frac{p}{1-p}\right) = ax + b$$

2. 로그(ln) 벗기기 양변에 밑이 e인 지수를 취하여 좌변의 로그를 벗겨냅니다. (이제 좌변은 오즈가 됩니다)

$$\frac{p}{1-p} = e^{ax+b}$$

3. 확률 p에 대하여 식 정리하기 양변에 (1-p)를 곱하고, p가 있는 항을 한쪽으로 모아 정리합니다.

$$p = \frac{e^{ax+b}}{1+e^{ax+b}}$$

4. 최종 형태 완성 (시그모이드 함수) 분모와 분자를 모두 e^{ax+b}로 나누어, 우리가 잘 아는 친숙한 시그모이드 형태로 만들어 줍니다.

$$p = \frac{1}{1+e^{-(ax+b)}}$$

로지스틱 회귀

- X 값으로 확률을 넣으면 logit(P) 꼴로 나타남
- 확률을 구하려면 기존 함수의 역함수를 취하여 연산

$$f(z) = y = -\log_e\left(\frac{1}{z} - 1\right)$$

$$z = -\log_e\left(\frac{1}{y} - 1\right)$$

$$e^{-z} = \frac{1-y}{y}$$

$$y \times e^{-z} + y = 1$$

$$y(e^{-z} + 1) = 1$$

$$y = \frac{1}{1+e^{-z}}$$

로지스틱 회귀

- 로지스틱 함수(logistic function) : 로짓 함수의 역함수
  - 그래프가 s자 커브 형태인 시그모이드 함수(sigmoid function)

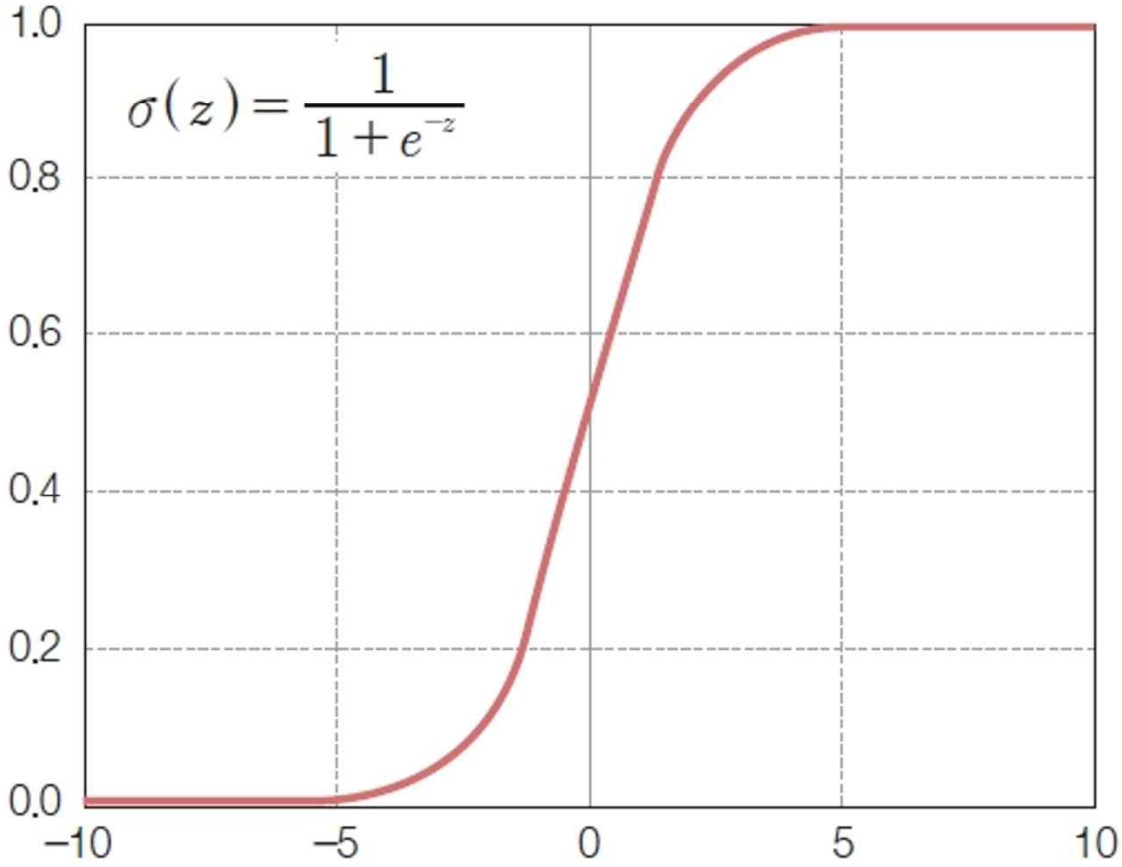


그림4. 시그모이드 함수

로지스틱 회귀

- 로지스틱 회귀(Logistic Regression) : 종속변수가 이분형일 때 수행할 수 있는, 예측 분석을 위한 회귀분석 기법
- 시그모이드 함수 수식
  - y 값을 확률 p로 표현
  - z 값은 선형회귀와 같이 가중치와 피쳐의 선형 결합(linear combination)으로 표현 가능

$$p = \sigma(z) = \frac{1}{1+e^{-z}}, \quad \frac{p}{1-p} = \frac{\frac{1}{1+e^{-z}}}{\frac{e^{-z}}{1+e^{-z}}} = \frac{1}{e^{-z}} = e^{z}$$

$$\log_e \frac{p}{1-p} = z$$

$$\log_e \frac{p}{1-p} = z = w_0 x_0 + w_1 x_1 + \cdots + w_n x_n$$

** **확률을 보고 싶으면 시그모이드**를 쓰고, **데이터와의 선형적 관계를 보고 싶으면 로짓**을.

로지스틱 회귀

시그모이드 함수

3-1.sample

```python
import numpy as np
import matplotlib.pyplot as plt

# [1단계] 시그모이드 함수 정의 만들기
def sigmoid(z):  4개의 사용 위치
    return 1 / (1 + np.exp(-z))
print(" [PART 1] 시그모이드 함수 수치 테스트 ")

# 1. 입력값이 딱 0일 때 (정확히 절반의 확률!)
s_0 = sigmoid(0)
print(f"입력값  0    => 압축된 확률: {s_0:.5f} (50%)")

 # 2. 아주 큰 양수일 때 (100% 확신에 수렴)
s_100 = sigmoid(100)
print(f"입력값  => 압축된 확률: {s_100:.5f} (약 100%)")

# 3. 아주 작은 음수일 때 (0% 확신에 수렴)
s_m100 = sigmoid(-100)
print(f"입력값 -100 => 압축된 확률: {s_m100:.5f} (약 0%)")

# [PART 2] 맷플롯립(matplotlib)으로 시그모이드 곡선 시각화하기
# 1. 그래프를 그릴 X축 데이터(날것의 점수 z) 만들기
z_values = np.linspace(-10, stop: 10, num: 200)

# 2. 만든 X축 데이터를 시그모이드 터널에 통과시키기 (Y축 확률값 획득!)
probabilities = sigmoid(z_values)

# 3. 그래프 크기 설정
plt.figure(figsize=(10, 6))

# 4. 시그모이드 S자 곡선 그리기 (빨간색 굵은 선)
plt.plot( *args: z_values, probabilities, color='red', linewidth=3, label='Sigmoid Curve')

# Y축 기준선 (상한선 1.0, 하한선 0.0)
plt.axhline(y=1.0, color='gray', linestyle='--', alpha=0.5)
plt.axhline(y=0.0, color='gray', linestyle='--', alpha=0.5)

# 50% 합격 기준선 (임곗값 0.5)
plt.axhline(y=0.5, color='black', linestyle=':', label='Threshold (0.5)')

# X축 기준선 (입력값 0)
plt.axvline(x=0.0, color='gray', linestyle='--', alpha=0.5)

# 아까 콘솔에서 테스트한 (0, 0.5) 지점에 파란 점 찍기
plt.scatter( x: 0,  y: 0.5, color='blue', s=100, zorder=5, label='sigmoid(0) = 0.5')

# 5. 그래프 꾸미기 (제목, 축 이름, 격자 무늬, 범례)
plt.title('Sigmoid Function (Magic Compressor)')
plt.xlabel('Raw Score (z) - from AI model')      # X축: AI 모델의 날것 점수
plt.ylabel('Probability (0.0 ~ 1.0) - output') # Y축: 압축된 확률값
plt.grid( visible: True, linestyle='-', alpha=0.3) # 배경 격자
plt.legend(loc='upper left')   # 범례를 왼쪽 위에 표시

# 6. 파이참에서 완성된 그래프 창 띄우기
plt.show()
```

2. 로지스틱 회귀의 기본 함수

2.1 가설함수

- 가설함수(hypothesis function)

$$h_\theta(x) = g(z) = \frac{1}{1+e^{-z}}$$

  - z는 가중치 값과 피쳐 값의 선형 결합
  - 가중치 값을 찾는 학습을 위해 경사하강법 알고리즘 사용

$$z = w_0x_0 + w_1x_1 + \cdots + w_nx_n = \theta^T X$$

로지스틱 회귀

가설함수
3-2.sample

```python
import numpy as np

# [1단계] 이전에 만든 시그모이드 함수
def sigmoid(z):  1개의 사용 위치
    return 1 / (1 + np.exp(-z))

# [2단계] 가설 함수 (Hypothesis Function) 정의
# 공식: h(x) = sigmoid(theta^T * x)
def hypothesis_function(x, theta):  1개의 사용 위치
    # np.dot은 행렬의 곱(내적)을 계산해주는 함수입니다.
    # z = theta[0]*x[0] + theta[1]*x[1] ... 와 같은 계산을 한 줄로 끝냅니다.
    z = np.dot(x, theta)
    return sigmoid(z)

# [3단계] 실제 데이터로 테스트
# 예: 온도(x1)=30도, 진동(x2)=0.8 일 때
x_data = np.array([30, 0.8])
theta_data = np.array([0.1, 5.0])

# 최종 확률 계산
prob = hypothesis_function(x_data, theta_data)

print(f"종합 점수(z): {np.dot(x_data, theta_data)}")
print(f"최종 고장 확률: {prob * 100:.2f}%")
```

2.2 비용함수

- 먼저 비용함수를 정의하고 예측값과 실제값 간의 차이를 최소화하는 방향으로 학습
- 실제값이 1일 때와 실제값이 0일 때 각각 다르게 비용함수를 정의

$$Cost(h_\theta(x), y) = \begin{cases} -\log(h_\theta(x)) & \text{if } y = 1 \\ -\log(1 - h_\theta(x)) & \text{if } y = 0 \end{cases}$$

로지스틱 회귀

- (a)는 y = 1일 때, (b)는 y = 0일 때 비용함수 그래프 (0 ≤ h ≤ 1)

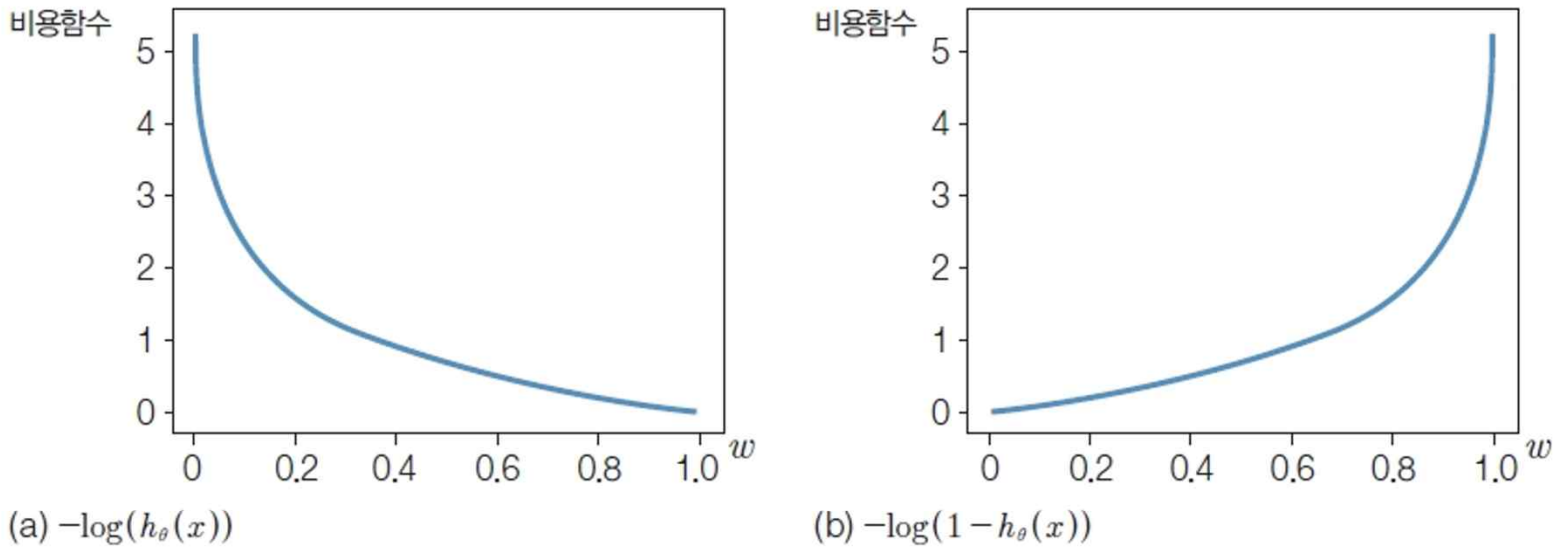


그림5. 비용함수 그래프

- (a)에서 h 값이 1에 가까워질수록 비용함수가 0에 가까워짐
  (b)에서 h 값이 0에 가까워질수록 비용함수가 0에 가까워짐

로지스틱 회귀

- 두 경우의 비용함수를 하나로 통합

$$J(\theta) = \frac{1}{m}\sum_{i=1}^{m} Cost(h_\theta(x^{(i)}), y^{(i)})$$

$$= -\frac{1}{m}\sum_{i=1}^{m}[y^{(i)}\log h_\theta(x^{(i)}) + (1-y^{(i)})\log(1-h_\theta(x^{(i)}))]$$

로지스틱 회귀

비용함수

3-3.sample

```python
import numpy as np

# [1단계] 시그모이드 함수
def sigmoid(z):  1개의 사용 위치
    return 1 / (1 + np.exp(-z))

# [2단계] 가설 함수 (Hypothesis Function)
def hypothesis_function(x, theta):  2개의 사용 위치
    z = np.dot(x, theta)
    return sigmoid(z)

# [3단계] 비용 함수 (Cost Function / Log Loss)
def compute_cost(x, y, theta):  1개의 사용 위치
    m = y.shape[0]  # 데이터의 개수
    h = hypothesis_function(x, theta)  # 모델의 예측 확률값

    # 로그 손실 공식 적용 (y=1일 때와 y=0일 때의 손실을 합산)
    term1 = y.T.dot(np.log(h))
    term2 = (1 - y).T.dot(np.log(1 - h))

    J = (-1.0 / m) * (term1 + term2)
    return J

# [4단계] 실제 데이터 대입 및 결과 출력 (실행부)
X = np.array([
    [1, 2],
    [3, 4],
    [5, 6]
])
# 실제 정답 (1: 고장, 0: 정상, 1: 고장)
y = np.array([1, 0, 1])

# 임의로 설정한 가중치 (AI의 현재 지식 수준)
theta = np.array([0.1, 0.2])

# 1. 먼저 예측 확률 확인하기
predictions = hypothesis_function(X, theta)
print("=== 1. 모델의 예측 확률 ===")
print(predictions)
print()

# 2. 이 예측이 얼마나 틀렸는지 비용 계산하기
cost = compute_cost(X, y, theta)
print("=== 2. 현재 모델의 비용(Cost) ===")
print(f"반성 점수: {cost:.4f}")
```

2.3 비용함수의 미분과 가중치 업데이트

- θ의 최적값을 구하기 위해 J 값을 θ에 대해 미분
  - θ는 z값 안에 있는, w_j의 집합

$$\frac{\partial}{\partial \theta_j} J(\theta) = \frac{1}{m} \sum_{i=1}^{m} (h_\theta(x^i) - y^i) x_j^i$$

- 가중치 값을 업데이트
  - 선형회귀와 동일하게 모든 θ에 대해 동시에 가중치가 업데이트됨

$$\theta_j = \theta_j - \alpha \frac{\partial}{\partial \theta} J(\theta)$$

$$= \theta_j - \alpha \sum_{i=1}^{m} (h_\theta(x^i) - y^i) x_j^i$$

로지스틱 회귀

가중치업데이트_경사하강법

3-4. sample

```python
import numpy as np

# [1단계] 기본 함수 셋팅 (시그모이드, 가설, 비용)
def sigmoid(z):  1개의 사용 위치
    return 1 / (1 + np.exp(-z))
def hypothesis_function(x, theta):  2개의 사용 위치
    z = np.dot(x, theta)
    return sigmoid(z)
def compute_cost(x, y, theta):  2개의 사용 위치
    m = y.shape[0]
    h = hypothesis_function(x, theta)
    # 안전한 로그 계산을 위해 아주 작은 값(1e-5)을 더해줍니다.
    term1 = y.T.dot(np.log(h + 1e-5))
    term2 = (1 - y).T.dot(np.log(1 - h + 1e-5))
    return (-1.0 / m) * (term1 + term2)

# [2단계] 경사하강법 함수 (가중치 업데이트)
def minimize_gradient(x, y, theta, iterations=1000, alpha=0.01):  1개의 사용 위치
    m = y.size
    cost_history = []
    for _ in range(iterations):
        # 예측값과 실제값의 차이(에러) 계산
        h = hypothesis_function(x, theta)
        loss = h - y
        # 기울기(Gradient) 계산: 에러와 데이터의 곱
        # x.T.dot(loss)는 모든 데이터에 대해 에러*데이터를 합산해줍니다.
        gradient = x.T.dot(loss) / m
        # 가중치 업데이트 (공식: theta = theta - 보폭 * 기울기)
        theta = theta - (alpha * gradient)
        # 100번마다 현재 비용(반성 점수) 기록
        if (_ % 100) == 0:
            current_cost = compute_cost(x, y, theta)
            cost_history.append(current_cost)
            print(f"반복 횟수 {_:>4} : 현재 비용(Cost) = {current_cost:.5f}")
    return theta, cost_history

# [3단계] 실제 데이터 대입 및 결과 확인 # 테스트용 데이터 (문제 3개, 특징 2개)
X_test = np.array([[1, 2], [3, 4], [5, 6]])
y_test = np.array([1, 0, 1])
initial_theta = np.array([0.0, 0.0])  # 초기 가중치 (아무것도 모르는 상태: 0, 0)

print("🚀 학습을 시작합니다...")# 학습 실행 (보폭 0.01로 1000번 반복)
final_theta, history = minimize_gradient(X_test, y_test, initial_theta, iterations=1000, alpha=0.01)
print("✅ 학습 완료!")
print(f"최종 가중치(theta): {final_theta}")
print(f"최종 비용(Cost): {compute_cost(X_test, y_test, final_theta):.5f}")
```

분류 문제에 있어서 성능지표의 필요성

- 모델을 평가하는 성능지표들
  - 회귀(regression) : MAE, MSE, RMSE, SSE
  - 분류(classification) : 정확도, 정밀도, 민감도, F1 스코어, ROC 커브, 리프트 차트
  - 클러스터링(clustering) : DBI, 엘보우 메서드, 실루엣계수

분류 문제의 성능지표

머신러닝에서 다양한 추가 성능지표를 사용하는 예

- 모델이 다른 모델보다 경제적으로 나은가?
- 모델이 사용하는 데이터가 많은가? 또는 적은가?
- 모델이 용량이 작은 컴퓨터에서도 작동하는가?
- 모델의 손해가 얼마나 나는가

혼동행렬

- 혼동행렬(confusion matrix): 예측값이 실제값 대비 얼마나 잘 맞는지 2×2 행렬로 표현
  - 일반적으로 질문의 '예'에 해당하는 값은 True 또는 1, '아니오'에 해당하는 값은 False 또는 0

		예측값(prediction)	
		1	0
실제값 (actual class)	1	True Positive	False Negative
	0	False Positive	True Negative

그림6. 혼동행렬

분류 문제의 성능지표

- 실제값과 예측값의 조합으로 발생 가능한 4가지 경우
  - True Positive(TP) : 예측값과 실제값이 모두 1로 동일할 때, 즉 모델의 예측값이 정답이고 예측 대상이 1일 때
  - True Negative(TN) : 예측값과 실제값이 모두 0으로 동일할 때, 즉 모델의 예측값이 정답이고 예측 대상이 0일 때
  - False Negative(FN) : 실제값은 1이지만 예측값이 0으로, 모델의 예측값이 오답이고 예측값이 0을 예측할 때
  - False Positive(FP) : 실제값은 0이지만 예측값이 1로, 모델의 예측값이 오답이고 예측값이 1을 예측할 때

분류 문제의 성능지표

혼동행렬표 나타내기 _3-1. practice

```python
# [1단계] 라이브러리 불러오기
from sklearn.metrics import confusion_matrix

# [2단계] 실험할 데이터 만들기
y_true = [1, 0, 1, 1, 0, 1]
y_pred = [0, 0, 1, 1, 0, 1]

# [3단계] 혼동 행렬(Confusion Matrix) 계산 및 출력
cm = confusion_matrix(y_true, y_pred)

print("1. 혼동 행렬 전체 모습")
print(cm)

# 👇 여기에 빈 print()를 추가하면 한 줄이 깔끔하게 띄워집니다! 👇
# (만약 두 줄을 띄우고 싶다면 print("\n") 이라고 적어주세요.)
print()

# [4단계] ravel() 함수로 2차원 행렬을 1차원으로 쫙 펴서 각각의 변수에 담기
tn, fp, fn, tp = confusion_matrix(y_true, y_pred).ravel()

print("2. 세부 지표 값 확인")
print(f"TN (True Negative) : {tn}")
print(f"FP (False Positive): {fp}")
print(f"FN (False Negative): {fn}")
print(f"TP (True Positive) : {tp}")
```

혼동행렬표를 사용한 지표

1. 정확도

- 정확도(accuracy) : 전체 데이터 개수 대비 정답을 맞춘 데이터의 개수

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

		예측값(prediction)	
		1	0
실제값 (actual class)	1	True Positive	False Negative
	0	False Positive	True Negative

그림7. 정확도 측정

분류 문제의 성능지표

정확도 구하기_3-2

```python
# [1단계] 필요한 라이브러리 불러오기
# 배열 계산을 위한 numpy와 정확도 계산을 위한 accuracy_score를 가져옵니다.
import numpy as np
from sklearn.metrics import accuracy_score

# [2단계] 실험할 데이터 만들기 (넘파이 배열 형태로 변환)
y_pred = np.array([0, 1, 1, 0]) # 모델이 예측한 값
y_true = np.array([0, 1, 0, 0]) # 실제 정답


# [3단계] 원리 이해하기: 넘파이(NumPy)를 이용해 '직접' 정확도 계산하기
manual_accuracy = sum(y_true == y_pred) / len(y_true)

print(" 1. 직접 계산한 정확도 ")
print(f"맞춘 개수(3) / 전체 개수(4) = {manual_accuracy}")
print( )


# [4단계] 실무 활용하기: 사이킷런(scikit-learn) 함수로 '한 번에' 계산하기
sklearn_accuracy = accuracy_score(y_true, y_pred)

print(" 2. 사이킷런 함수(accuracy_score)를 사용한 정확도 ")
print(f"결과: {sklearn_accuracy}")
```

분류 문제의 성능지표

2. 정밀도, 민감도, F1 스코어

- 정밀도와 민감도는 불균일한 데이터셋을 다룰 때 유용
  - 데이터에서 1과 0의 비율이 7:3 또는 3:7 이상 차이나는 상태
- 정밀도(precision) : 모델이 1이라고 예측했을 때 얼마나 잘 맞을지에 대한 비율

$$PRECISION(PPV) = \frac{TP}{TP + FP}$$

		예측값(prediction)	
		1	0
실제값 (actual class)	1	True Positive	False Negative
	0	False Positive	True Negative

그림8. 정밀도 측정

분류 문제의 성능지표

- 민감도(recall) : 실제 1인 값을 가진 데이터를 모델이 얼마나 1이라고 잘 예측했는지에 대한 비율
  - 반환율 또는 재현율이라고도 부름

$$RECALL(TPR) = \frac{TP}{TP + FN}$$

		예측값(prediction)	
		1	0
실제값 (actual class)	1	True Positive	False Negative
	0	False Positive	True Negative

그림9. 민감도 측정

분류 문제의 성능지표

- F1 스코어(F1 score) : 정밀도와 민감도의 조화평균 값

$$F_1 = 2 \times \frac{precision \times recall}{precision + recall}$$

분류 문제의 성능지표

정밀도, 민감도, F1 스코어 구하기_3-3

```python
# [1단계] 도구 챙기기: 넘파이와 3가지 평가 지표 함수 불러오기
import numpy as np
from sklearn.metrics import precision_score, recall_score, f1_score

# [2단계] 실험할 데이터 만들기
y_pred = np.array([0, 1, 1, 0, 1, 1, 1, 0]) # 모델의 예측값
y_true = np.array([0, 1, 0, 0, 0, 0, 1, 1]) # 실제 정답

# ---------------------------------------------------------
# [수동 계산을 위한 중간 점검]
# 데이터 8개를 하나씩 비교해보면 다음과 같습니다.
# TP (실제 1, 예측 1) = 2개
# FP (실제 0, 예측 1) = 3개 (오경보)
# FN (실제 1, 예측 0) = 1개 (놓침)
# TN (실제 0, 예측 0) = 2개
# ---------------------------------------------------------

# [3단계] 정밀도(Precision) 계산
# 공식: TP / (TP + FP) = 2 / (2 + 3) = 0.4
precision = precision_score(y_true, y_pred)
print("1. 정밀도(Precision)")
print(f"결과: {precision}")
print() # 줄 띄우기

# [4단계] 민감도/재현율(Recall) 계산
# 공식: TP / (TP + FN) = 2 / (2 + 1) = 0.666...
recall = recall_score(y_true, y_pred)
print("2. 민감도(Recall)")
print(f"결과: {recall}")
print()

# [5단계] F1 스코어(F1-score) 계산
# 정밀도와 민감도의 조화평균! 둘 중 하나라도 너무 낮으면 F1 점수도 깎입니다.
f1 = f1_score(y_true, y_pred)
print("3. F1 스코어(F1-score)")
print(f"결과: {f1}")
```

분류 문제의 성능지표

- 성능 측정

ROC 곡선

- 수신기 조작 특성(receiver operating characteristic, ROC) 곡선도
  - 거짓 양성 비율(false positive rate, FPR)에 대한 진짜 양성 비율(true positive rate, TPR, 재현율의 다른 이름)의 곡선
  - FPR 또는 폴-아웃(fall-out)은 양성으로 잘못 분류된 음성 샘플의 비율
    - 1에서 음성으로 정확하게 분류한 음성 샘플의 비율인 진짜 음성 비율(true negative rate, TNR)을 뺀 값
    - TNR – 특이도(specificity)라고도 함
  - ROC 곡선은 민감도(재현율)에 대한 1-특이도 그래프
  - roc_curve() 함수를 사용해 여러 임곗값에서 TPR과 FPR을 계산

```
from sklearn.metrics import roc_curve

fpr, tpr, thresholds = roc_curve(y_train_5, y_scores)
```

분류 문제의 성능지표

- 성능 측정
  - 맷플롯립을 사용해 TPR에 대한 FPR 곡선 구현

```
idx_for_threshold_at_90 = (thresholds <= threshold_for_90_precision).argmax()
tpr_90, fpr_90 = tpr[idx_for_threshold_at_90], fpr[idx_for_threshold_at_90]

plt.plot(fpr, tpr, linewidth=2, label="ROC 곡선")
plt.plot([0, 1], [0, 1], 'k:', label="랜덤 분류기의 ROC 곡선")
plt.plot([fpr_90], [tpr_90], "ko", label="90% 정밀도에 대한 임곗값")
[...] # 레이블, 그리드, 범례, 화살표, 텍스트를 추가합니다.
plt.show()
```

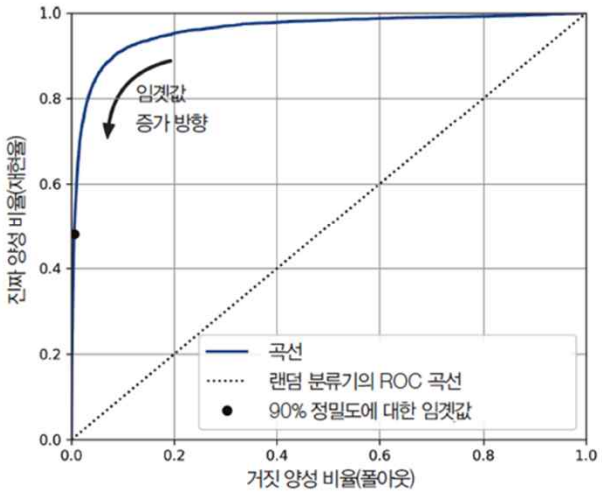


그림7 가능한 모든 임곗값에서 진짜 양성 비율에 대한 거짓 양성 비율을 나타낸 ROC 곡선 검은 점이 선택한 비율의 지점(90%의 정밀도와 48%의 재현율)

분류 문제의 성능지표

ROC 곡선 수치 계산

```python
# [1단계] 필요한 라이브러리 불러오기
import numpy as np
from sklearn.metrics import roc_curve
import matplotlib.pyplot as plt

# [2단계] 실험할 데이터 만들기
y_true = np.array([0, 0, 1, 1])
y_scores = np.array([0.1, 0.4, 0.35, 0.8])

# [3단계] roc_curve 함수를 사용해 여러 임곗값에서의 FPR, TPR 계산하기
fpr, tpr, thresholds = roc_curve(y_true, y_scores)

# [4단계] 계산된 결과 출력하기
print("=== ROC 곡선 계산 결과 ===")
print(f"기준점(Thresholds) : {thresholds}")
print(f"X축 - FPR (오경보 비율) : {fpr}")
print(f"Y축 - TPR (민감도)      : {tpr}")
print( )

# [5단계] 맷플롯립(matplotlib)을 이용해 ROC 곡선 그리기
plt.figure(figsize=(6, 6)) # 그래프의 가로세로 크기를 지정합니다.

# 1. 우리의 AI 모델 성능 선 그리기 (파란색 선과 동그라미 마커)
plt.plot( *args: fpr, tpr, marker='o', color='blue', linewidth=2, label='Our AI Model')

# 2. 랜덤 분류기의 기준선 그리기 (회색 점선)
plt.plot( *args: [0, 1], [0, 1], linestyle='--', color='gray', label='Random (50%)')

# 3. 그래프 꾸미기 (제목, 축 이름, 격자 무늬, 범례)
plt.title('ROC Curve Analysis')
plt.xlabel('False Positive Rate (FPR)') # X축: 가짜 알람 비율
plt.ylabel('True Positive Rate (TPR)')  # Y축: 진짜 알람 비율
plt.grid(True) # 배경에 보기 쉽게 눈금선(격자)을 넣습니다.
plt.legend()   # 어떤 선이 어떤 모델인지 알려주는 범례 표기

# 4. 파이참에서 완성된 그래프 창 띄우기 (필수!)
plt.show()
```